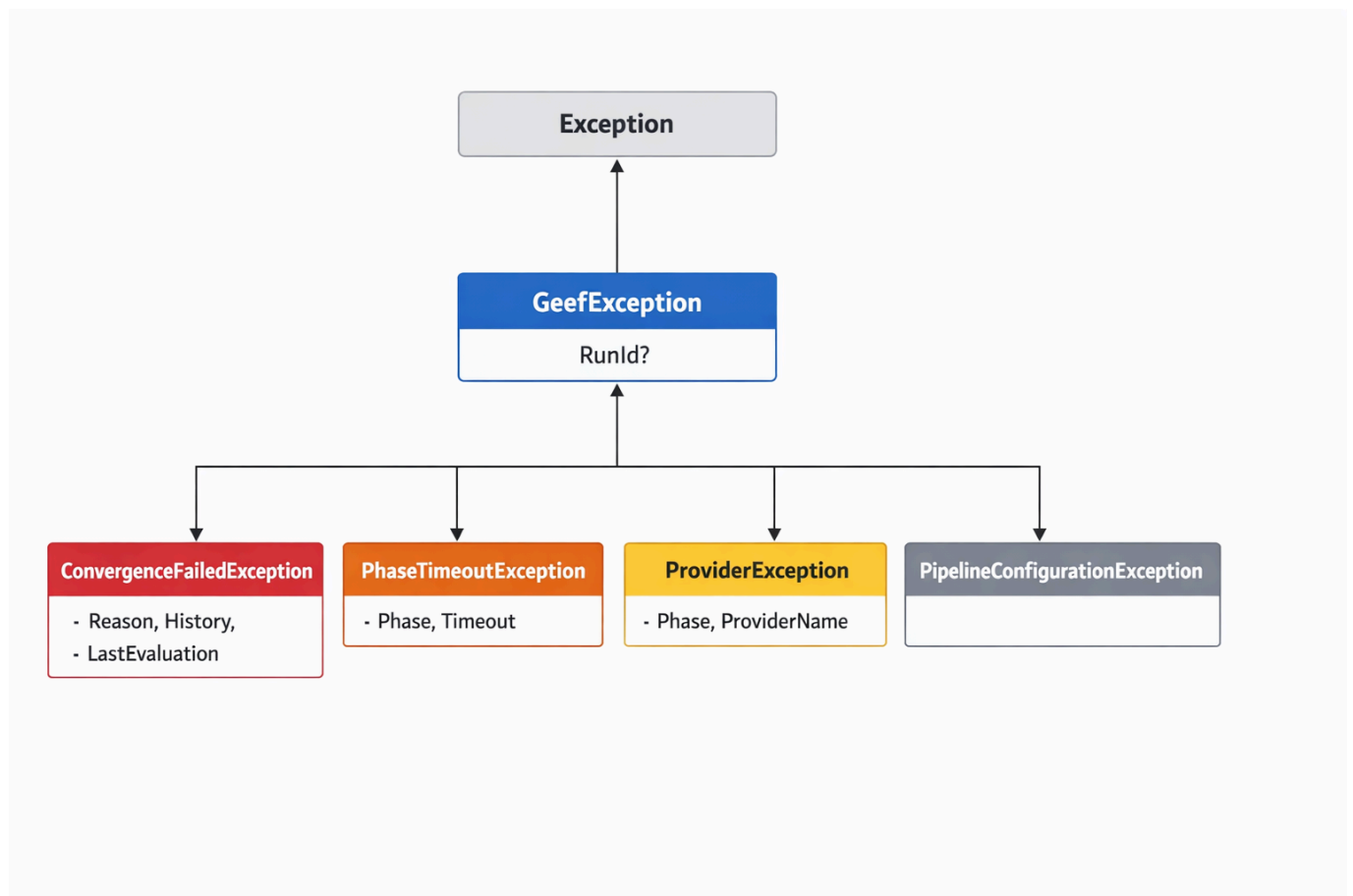


Fehlerbehandlung

Exception-Hierarchie



Alle GEEF-Exceptions erben von `GeefException` und tragen optional eine `RunId` zur Korrelation.

Fehlertypen im Detail

PipelineConfigurationException

Wann: Bei `Build()`, wenn Pflicht-Komponenten fehlen.

```

try
{
    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new MyGrounding())
        // Execution fehlt!
        .AddReviewer(new MyReviewer())
        .UseFinalizer(new MyFinalizer())
        .Build();
}
catch (PipelineConfigurationException ex)
{
    // "Execution step is required. Call UseExecution()."
    Console.WriteLine(ex.Message);
}

```

Pflicht-Checks:

- Grounding muss gesetzt sein
- Execution muss gesetzt sein
- Finalizer muss gesetzt sein
- Mindestens ein Reviewer muss registriert sein

ProviderException

Wann: Ein Provider (Grounding, Execution, Reviewer, Finalizer) wirft eine unerwartete Exception.

```

try
{
    var result = await pipeline.RunAsync("input");
}
catch (ProviderException ex)
{
    Console.WriteLine($"Phase: {ex.Phase}"); // z.B. GeefPhase.Execution
    Console.WriteLine($"Provider: {ex.ProviderName}"); // z.B. "MyExecution"
    Console.WriteLine($"RunId: {ex.RunId}");
    Console.WriteLine($"Ursache: {ex.InnerException}");
}

```

Wrapping-Verhalten:

- `OperationCanceledException` wird **nicht** gewrapped (durchgereicht)
- `GeefException` wird **nicht** gewrapped (kein Double-Wrapping)

- Alle anderen Exceptions werden in `ProviderException` gewrapped

PipelineFailedEvent: Vor jedem `throw ProviderException` wird ein `PipelineFailedEvent` publiziert.

ConvergenceFailedException

Wann: Die Konvergenz-Policy gibt eine terminale Entscheidung zurück — das umfasst `StopMaxAttemptsReached`, `StopStagnant`, `StopRegression`, `AbortCriticalBlocker` und `EscalateToHuman`.

```
try
{
    var result = await pipeline.RunAsync("input");
}
catch (ConvergenceFailedException ex)
{
    Console.WriteLine($"Grund: {ex.Reason}");           // z.B. StopMaxAttemptsReached
    Console.WriteLine($"Iterationen: {ex.History.Count}");

    // Letzte Findings inspizieren
    foreach (var finding in ex.LastEvaluation.AllFindings)
        Console.WriteLine($" [{finding.Severity}] {finding.Message}");
}
```

PhaseTimeoutException

Wann: Eine Phase überschreitet ihr Timeout (erfordert `TimeoutMiddleware`).

```
try
{
    var result = await pipeline.RunAsync("input");
}
catch (PhaseTimeoutException ex)
{
    Console.WriteLine($"Phase: {ex.Phase}");           // z.B. GeefPhase.Execution
    Console.WriteLine($"Timeout: {ex.Timeout}");       // z.B. 00:05:00
}
```

Reviewer-Fehler vs. Provider-Fehler

Es gibt einen wichtigen Unterschied:

Reviewer liefert **Failed**

Ein Reviewer, der technisch versagt (z.B. API-Timeout), sollte `ReviewDecision.Failed` zurückgeben — **nicht** eine Exception werfen:

```
public async Task<ReviewResult> ReviewAsync(IRunContext context, CancellationToken ct)
{
    try
    {
        var response = await _httpClient.PostAsync("api/review", content, ct);
        // ... Ergebnis auswerten
    }
    catch (HttpRequestException ex)
    {
        return new ReviewResult
        {
            ReviewerName = Name,
            Decision = ReviewDecision.Failed,
            Findings = new[]
            {
                new Finding
                {
                    ReviewerName = Name,
                    Fingerprint = "api-timeout",
                    Message = $"Review-API nicht erreichbar: {ex.Message}",
                    Severity = FindingSeverity.Error
                }
            },
            Duration = TimeSpan.Zero
        };
    }
}
```

Reviewer wirft Exception

Wenn ein Reviewer eine unbehandelte Exception wirft, wird sie vom Runtime als `ProviderException` gewrapped. Der Loop wird sofort abgebrochen. Die `ProviderException` hat `Phase = GeefPhase.Evaluation` und `ProviderName` wird auf den Typ-Namen der `IEvaluationStrategy` gesetzt (z.B. `"SequentialEvaluationStrategy"`), nicht auf den individuellen Reviewer.

Empfehlung: Fangen Sie erwartete Fehler im Reviewer ab und geben Sie `ReviewDecision.Failed` zurück. Werfen Sie nur bei wirklich fatalen Fehlern (z.B. `OutOfMemoryException`).

Cancellation

Externes Cancellation

```
using var cts = new CancellationTokenSource(TimeSpan.FromMinutes(10));

try
{
    var result = await pipeline.RunAsync("input", cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("Pipeline wurde abgebrochen");
}
```

Kooperatives Cancellation

Alle Provider-Interfaces akzeptieren ein `CancellationToken`:

```
public async Task<ExecutionResult> RunAsync(IRunContext context, CancellationToken ct)
{
    ct.ThrowIfCancellationRequested();

    var result = await _llmClient.GenerateAsync(prompt, ct);

    ct.ThrowIfCancellationRequested();

    return new ExecutionResult { UpdatedContext = context.Set(key, result) };
}
```

Middleware-Cancellation

`TimeoutMiddleware` erstellt ein verknüpftes `CancellationToken` und setzt es auf `GeefMiddlewareContext.CancellationToken`. Das bedeutet:

- Downstream-Middleware und die Operation sehen den timeout-fähigen Token
- Bei Timeout: `OperationCanceledException` wird als `PhaseTimeoutException` gewrapped
- Bei externem Cancel: `OperationCanceledException` wird durchgereicht

Best Practices

1. Immer `RunId` **loggen**: Nutzen Sie `ex.RunId` zur Korrelation mit Events und Tracing.

2. **Reviewer-Fehler abfangen:** Geben Sie `Failed` zurück statt Exceptions zu werfen.
3. **CancellationToken weiterreichen:** Alle async-Aufrufe sollten das Token akzeptieren.
4. **ConvergenceFailedException auswerten:** Die `History` enthält wertvolle Diagnose-Informationen.
5. **PipelineFailedEvent abonnieren:** Für Alerting bei Produktions-Pipelines.